



Università degli Studi di Salerno
Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

Majority Dynamics in Online Networks

Docente/i

Prof.ssa Adele Anna Rescigno
Prof.ssa Luisa Gargano

Studente

Andrea Gisolfi
Matr. 0522502044

Anno Accademico 2025-2026

Indice

1	Introduzione	2
2	Metodologia	4
2.1	La rete utilizzata per gli esperimenti	4
2.2	Majority Cascade in Networks	6
2.3	Funzioni di costo	7
2.3.1	Random Cost Function	8
2.3.2	Degree Cost Function	8
2.4	Algoritmi utilizzati	9
2.4.1	Cost-Seeds-Greedy con f1	9
2.4.2	Cost-Seeds-Greedy con f2	10
2.4.3	Cascade-Gain Greedy	11
2.5	Protocollo sperimentale	13
3	Risultati	16
3.1	Influenza al variare del budget	16
3.1.1	Random Cost Function	17
3.1.2	Degree Cost Function	18
3.2	Confronto complessivo	19
3.3	Analisi dei tempi di esecuzione	20
3.4	Valutazione della robustezza	20
3.4.1	Rimozione casuale di archi	21
3.4.2	Rimozione casuale di nodi	22
3.5	Discussione finale	24
4	Conclusioni	26

Elenco delle figure

2.1	Distribuzione dei gradi dei nodi della rete Facebook.	5
2.2	Distribuzione del coefficiente di clustering locale nella rete Facebook.	6
3.1	Numero di nodi influenzati al variare del budget con Random Cost Function.	17
3.2	Numero di nodi influenzati al variare del budget con Degree Cost Function.	18
3.3	Robustezza alla rimozione casuale di archi con Random Cost Function. Le barre di errore rappresentano la deviazione standard sulle 20 ripetizioni.	21
3.4	Robustezza alla rimozione casuale di archi con Degree Cost Function. Le barre di errore rappresentano la deviazione standard sulle 20 ripetizioni.	21
3.5	Robustezza dei seed set alla rimozione casuale di nodi con Random Cost Function. Le barre di errore rappresentano la deviazione standard sulle 20 ripetizioni.	23
3.6	Robustezza dei seed set alla rimozione casuale di nodi con Degree Cost Function. Le barre di errore rappresentano la deviazione standard sulle 20 ripetizioni.	23

Elenco delle tabelle

2.1	Principali caratteristiche strutturali della rete Facebook utilizzata negli esperimenti.	5
3.1	Percentuale di nodi influenzati al variare del budget con Random Cost Function.	17
3.2	Percentuale di nodi influenzati al variare del budget con Degree Cost Function.	19
3.3	Tempo medio di esecuzione dei tre algoritmi.	20
3.4	Retention media (%) dopo la rimozione del 20% degli archi.	22
3.5	Retention media (%) dopo la rimozione del 20% degli nodi.	24

Capitolo 1

Introduzione

Le reti sociali online costituiscono sistemi dinamici nei quali le relazioni tra gli utenti possono modificarsi continuamente nel tempo. Nuovi collegamenti possono essere creati, relazioni esistenti possono essere interrotte e gli stessi utenti possono entrare o uscire dalla rete. Tali variazioni strutturali rendono particolarmente interessante lo studio dei processi di **diffusione dell'influenza**, attraverso i quali informazioni, opinioni o comportamenti possono propagarsi da un insieme iniziale di individui al resto della rete.

Un problema centrale nell'ambito dell'*Influence Diffusion* consiste nell'individuare un opportuno insieme di nodi inizialmente attivi, denominato **seed set**, capace di generare una diffusione quanto più ampia possibile. L'efficacia della propagazione dipende sia dalla posizione dei seed all'interno della rete sia dal modello utilizzato per descrivere il processo di attivazione dei nodi.

In questo progetto viene considerato un processo di diffusione basato sulla **Majority Dynamics**. Data una rete rappresentata mediante un grafo non orientato

$$G = (V, E),$$

un nodo inizialmente inattivo diventa attivo quando almeno metà dei propri vicini risulta già attiva. Il processo evolve in round successivi fino al raggiungimento di una configurazione stabile, nella quale nessun ulteriore nodo può essere attivato. Una caratteristica fondamentale del modello adottato è la sua monotonicità: una volta attivato, un nodo rimane attivo per tutta la durata della diffusione.

Indicando con $S \subseteq V$ il seed set iniziale, l'insieme complessivo dei nodi attivati al termine del processo viene indicato con

$$Inf[G, S].$$

L'obiettivo generale consiste quindi nell'individuare seed set capaci di massimizzare

$$|Inf[G, S]|.$$

Il problema viene ulteriormente generalizzato introducendo un costo associato a ciascun nodo della rete. Sia

$$c : V \rightarrow \mathbb{N}$$

una funzione che assegna a ogni nodo $u \in V$ un costo $c(u)$. Il costo complessivo di un seed set è definito come

$$c(S) = \sum_{u \in S} c(u).$$

Dato un budget massimo k , la selezione dei seed deve quindi rispettare il vincolo

$$c(S) \leq k,$$

cercando contemporaneamente di massimizzare il numero di nodi influenzati. L'individuazione della soluzione ottima è un problema computazionalmente difficile; per tale motivo il progetto si concentra sull'utilizzo e sul confronto sperimentale di differenti **strategie euristiche di selezione dei seed**.

In particolare, vengono analizzate due varianti dell'algoritmo **Cost-Seeds-Greedy**, basate su due differenti funzioni di valutazione marginale, e un terzo approccio sviluppato appositamente per il progetto, denominato **Cascade-Gain Greedy**. Quest'ultimo utilizza direttamente l'incremento effettivo della Majority Cascade prodotto dall'aggiunta di un candidato al seed set, con l'obiettivo di privilegiare i nodi che generano il maggiore aumento reale della diffusione per unità di costo.

Il confronto tra gli algoritmi viene effettuato considerando differenti funzioni di costo e diversi livelli di budget, in modo da osservare come la quantità di risorse disponibili influenzi la capacità del seed set di attivare la rete.

Un ulteriore obiettivo del progetto consiste nello studiare la **robustezza** delle soluzioni rispetto alla natura dinamica delle online networks. Una volta determinato un seed set sulla rete originale G , vengono simulate modifiche strutturali attraverso la rimozione casuale di archi e di vertici, ottenendo una nuova rete G' . Il seed set determinato prima della perturbazione non viene ricalcolato; viene invece valutata la sua capacità di mantenere efficacia nella rete modificata, confrontando

$$|Inf[G, S]|$$

con la diffusione ottenuta dopo la perturbazione.

L'analisi sperimentale permette quindi di osservare il problema da due prospettive complementari. La prima riguarda l'**efficacia iniziale** delle diverse strategie di selezione, misurata attraverso il numero di nodi raggiunti dalla cascade sotto diversi vincoli di budget. La seconda riguarda la **stabilità delle soluzioni**, valutando quanto l'influenza prodotta dai seed selezionati sia sensibile ai cambiamenti nella struttura della rete.

Il lavoro è organizzato come segue. Il Capitolo 2 descrive la metodologia adottata, presentando la rete utilizzata, il modello di Majority Cascade, le funzioni di costo, gli algoritmi considerati e il protocollo sperimentale. Il Capitolo 3 presenta e discute i risultati ottenuti, analizzando l'effetto del budget, il comportamento delle differenti strategie, i tempi di esecuzione e la robustezza rispetto alla rimozione di archi e nodi. Infine, il Capitolo 4 riassume le principali evidenze emerse e presenta le conclusioni del lavoro.

Capitolo 2

Metodologia

Questo capitolo descrive la metodologia adottata per lo sviluppo e la valutazione sperimentale del progetto. In particolare, vengono presentati la rete utilizzata per gli esperimenti, il modello di Majority Cascade, le funzioni di costo considerate, gli algoritmi di selezione del seed set e il protocollo utilizzato per confrontarne efficacia e robustezza. L'analisi viene condotta sia sulla rete originale sia su versioni perturbate ottenute mediante rimozione casuale di archi e nodi.

L'implementazione completa degli algoritmi, delle procedure sperimentali e degli script utilizzati per la generazione dei risultati è disponibile nella repository GitHub del progetto al seguente indirizzo: <https://github.com/gisolfi02/Majority-Dynamics>.

2.1 La rete utilizzata per gli esperimenti

Per la valutazione sperimentale è stata utilizzata una rete sociale reale appartenente al dataset **Facebook Social Circles**¹, distribuito all'interno della collezione **Stanford Network Analysis Project (SNAP)**. In particolare, è stata considerata la rete aggregata contenuta nel file *facebook_combined.txt*, che rappresenta relazioni di amicizia tra utenti Facebook anonimizzati.

La rete viene modellata mediante un grafo non orientato

$$G = (V, E),$$

nel quale ciascun vertice $v \in V$ rappresenta un utente e ciascun arco $(u, v) \in E$ rappresenta una relazione di amicizia tra due utenti. La natura non orientata del grafo risulta coerente con il tipo di relazione considerata, poiché un collegamento di amicizia viene interpretato come reciproco.

Le principali proprietà strutturali della rete sono riportate nella Tabella 2.1.

¹<https://snap.stanford.edu/data/ego-Facebook.html>

Tabella 2.1: Principali caratteristiche strutturali della rete Facebook utilizzata negli esperimenti.

Proprietà	Valore
Numero di nodi	4039
Numero di archi	88234
Grado medio	43.69
Grado minimo	1
Grado massimo	1045
Densità	0.01082
Numero di componenti connesse	1
Dimensione componente gigante	4039
Coefficiente medio di clustering	0.6055
Diametro	8

Per caratterizzare più nel dettaglio la struttura della rete sono state analizzate la **distribuzione dei gradi** e la **distribuzione del coefficiente di clustering locale** (Figura 2.1 e Figura 2.2). La prima permette di osservare come il numero di connessioni sia distribuito tra gli utenti, mentre la seconda fornisce un'indicazione della tendenza dei vicini di un nodo a essere collegati anche tra loro.

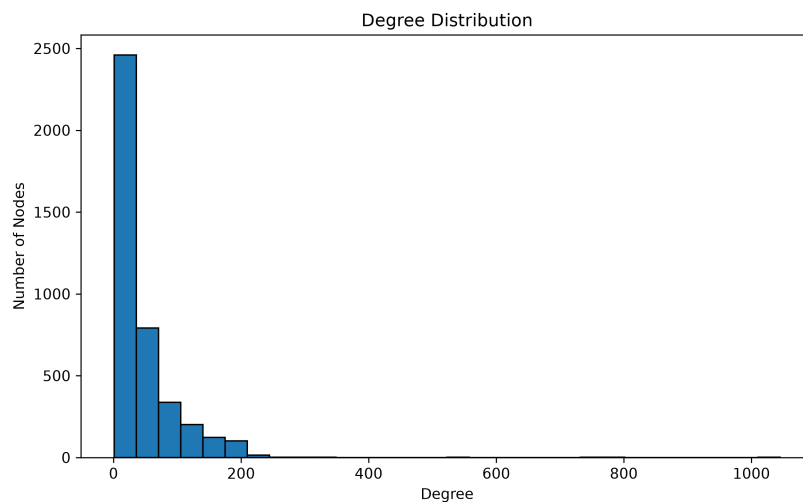


Figura 2.1: Distribuzione dei gradi dei nodi della rete Facebook.

La distribuzione del grado mostra una struttura fortemente asimmetrica: la maggior parte dei nodi possiede un numero relativamente contenuto di connessioni, mentre un numero molto ridotto di nodi presenta un grado particolarmente elevato. Questi ultimi possono essere interpretati come nodi fortemente connessi, o *hub*, all'interno della rete.

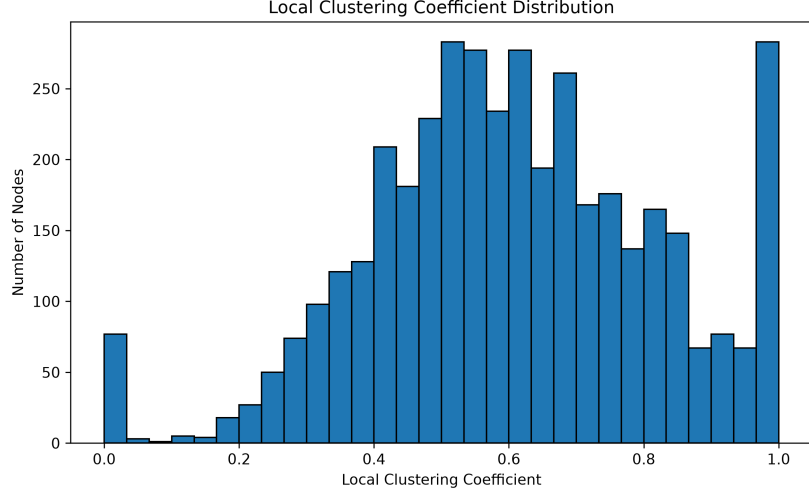


Figura 2.2: Distribuzione del coefficiente di clustering locale nella rete Facebook.

La distribuzione del coefficiente di clustering locale mostra invece che gran parte dei nodi presenta valori medio-alti, prevalentemente compresi tra circa 0.4 e 0.8, con una presenza significativa anche di valori prossimi a 1. Ciò indica che, per molti nodi, una parte consistente dei rispettivi vicini è a sua volta collegata tra loro.

La scelta di questa rete è quindi motivata sia dalla sua natura di **online social network reale**, direttamente coerente con il problema affrontato, sia dalle sue dimensioni. Il numero di nodi e archi è sufficientemente elevato da consentire un'analisi significativa dei fenomeni di diffusione, ma al tempo stesso permette di eseguire ripetutamente gli algoritmi di selezione e le simulazioni di perturbazione con costi computazionali gestibili.

2.2 Majority Cascade in Networks

Il processo di diffusione considerato nel progetto è basato sul modello di **Majority Cascade**, nel quale lo stato di ciascun nodo dipende dal numero di vicini già attivi. Data una rete non orientata

$$G = (V, E)$$

e un insieme iniziale di nodi attivi

$$S \subseteq V,$$

detto **seed set**, il processo evolve attraverso una sequenza discreta di round:

$$Inf[S, 0], Inf[S, 1], \dots, Inf[S, r], \dots \subseteq V.$$

All'istante iniziale risultano attivi esclusivamente i nodi appartenenti al seed set:

$$Inf[S, 0] = S.$$

Per ogni nodo $v \in V$, indicando con $N(v)$ il suo insieme di vicini e con

$$d(v) = |N(v)|$$

il relativo grado, l'attivazione avviene quando almeno metà dei suoi vicini risulta già attiva. Poiché il numero di vicini attivi è necessariamente intero, nell'implementazione la soglia di attivazione viene espressa come

$$\tau(v) = \left\lceil \frac{d(v)}{2} \right\rceil$$

e il nodo v può quindi attivarsi quando

$$|N(v) \cap \text{Inf}[S, r-1]| \geq \tau(v).$$

L'insieme dei nodi attivi al round r è pertanto definito come

$$\text{Inf}[S, r-1] \cup \{v \in V \setminus \text{Inf}[S, r-1] : |N(v) \cap \text{Inf}[S, r-1]| \geq \tau(v)\}.$$

Il modello adottato è **monotono**: un nodo che diventa attivo a un determinato round rimane attivo in tutti i round successivi. Ne consegue che

$$\text{Inf}[S, r-1] \subseteq \text{Inf}[S, r]$$

per ogni r , e il processo termina quando viene raggiunta una configurazione stabile, ossia nel più piccolo istante t tale che

$$\text{Inf}[S, t] = \text{Inf}[S, t+1].$$

L'insieme finale dei nodi raggiunti dalla diffusione viene indicato con

$$\text{Inf}[G, S] = \text{Inf}[S, t]$$

e la quantità

$$|\text{Inf}[G, S]|$$

costituisce la principale misura di efficacia di un seed set utilizzata negli esperimenti.

2.3 Funzioni di costo

Nel problema considerato, a ciascun nodo della rete viene associato un costo attraverso una funzione

$$c \rightarrow \mathbb{N}.$$

Il costo rappresenta la quantità di risorse necessaria per includere un nodo nel seed set. Dato un insieme di seed

$$S \subseteq V,$$

il suo costo complessivo è definito come

$$c(S) = \sum_{u \in S} c(u)$$

e deve rispettare il vincolo di budget

$$c(S) \leq k.$$

La funzione di costo assume quindi un ruolo centrale nella selezione dei seed: due nodi potenzialmente simili dal punto di vista della capacità di influenza possono essere valutati in maniera differente se richiedono quantità diverse di budget.

Per valutare il comportamento degli algoritmi in condizioni differenti sono state considerate due funzioni di costo: una indipendente dalle caratteristiche strutturali del nodo e una direttamente dipendente dal suo grado.

2.3.1 Random Cost Function

La prima funzione di costo considerata assegna a ciascun nodo della rete un valore intero generato casualmente all'interno di un intervallo prefissato. In accordo con la formulazione del progetto, viene quindi definita una funzione

$$c_{\text{random}} \rightarrow \mathbb{N},$$

tale che, per ogni nodo $u \in V$,

$$c_{\text{random}}(u) \in [1, 10]$$

con valore scelto casualmente.

L'utilizzo di una funzione di costo casuale permette di modellare uno scenario nel quale il costo associato alla selezione di un nodo non dipende direttamente dalle sue caratteristiche strutturali, come il grado o la posizione nella rete. Due nodi con proprietà topologiche molto differenti possono quindi presentare costi simili, mentre nodi strutturalmente simili possono avere costi diversi.

Per garantire la **riproducibilità degli esperimenti**, la generazione dei costi viene effettuata mediante un generatore pseudo-casuale inizializzato con un seed fissato. In questo modo, a parità di seed, ogni nodo riceve sempre lo stesso costo in tutte le esecuzioni del programma, questo è fondamentale per garantire un confronto corretto tra gli algoritmi.

2.3.2 Degree Cost Function

La seconda funzione di costo considerata assegna a ciascun nodo un costo proporzionale al proprio grado. L'idea è quindi quella di rendere più costosa la selezione dei nodi maggiormente connessi, che in una rete sociale possono rappresentare utenti potenzialmente più influenti.

Per ogni nodo $u \in V$, indicando con

$$d(u) = |N(u)|$$

il suo grado, la funzione di costo viene definita come

$$c_{\text{degree}}(u) = \left\lceil \frac{d(u)}{2} \right\rceil$$

A differenza della funzione Random, questa funzione introduce quindi una relazione diretta tra struttura della rete e costo di selezione. I nodi con pochi collegamenti risultano relativamente economici, mentre i nodi ad alto grado richiedono una quota maggiore del budget.

2.4 Algoritmi utilizzati

Per la selezione del seed set sono stati considerati tre approcci euristici, confrontati a parità di rete, funzione di costo e budget disponibile.

I primi due derivano dall'algoritmo **Cost-Seeds-Greedy**, applicato rispettivamente alle funzioni obiettivo f_1 e f_2 . In entrambi i casi, la selezione dei nodi avviene in modo iterativo valutando il rapporto tra il guadagno marginale prodotto dall'inserimento di un candidato e il relativo costo.

Il terzo approccio, denominato **Cascade-Gain Greedy**, è stato invece sviluppato nell'ambito del progetto con l'obiettivo di valutare direttamente l'incremento reale della Majority Cascade generato dall'aggiunta di ciascun nodo al seed set corrente.

2.4.1 Cost-Seeds-Greedy con f_1

Il primo algoritmo considerato è Cost-Seeds-Greedy, utilizzato con la funzione obiettivo f_1 . L'algoritmo costruisce il seed set in maniera iterativa, partendo dall'insieme vuoto e selezionando, a ogni passo, il nodo che offre il miglior compromesso tra il beneficio marginale prodotto e il relativo costo.

Sia S il seed set corrente e sia $u \notin S$ un possibile candidato. Il guadagno marginale associato all'inserimento di u è definito come

$$f_1(S \cup u) - f_1(S).$$

La selezione avviene quindi sulla base del rapporto

$$\frac{\Delta_u f_1(S)}{c(u)}$$

e viene scelto il nodo

$$\arg \max_{u \in V \setminus S} \frac{\Delta_u f_1(S)}{c(u)}.$$

In questo modo l'algoritmo privilegia i nodi capaci di produrre il maggiore incremento della funzione obiettivo per unità di costo.

La funzione f_1 è definita come

$$\sum_{v \in V} \min \left\{ |N(v) \cap S|, \left\lceil \frac{d(v)}{2} \right\rceil \right\}$$

dove $N(v)$ rappresenta il vicinato del nodo v e

$$\left\lceil \frac{d(v)}{2} \right\rceil$$

è la relativa soglia majority.

Per ciascun nodo v , la funzione considera quindi il numero di suoi vicini già appartenenti al seed set, limitando però tale contributo alla soglia di attivazione. Una volta raggiunto il valore $\tau(v)$, l'aggiunta di ulteriori seed nel vicinato di v non produce alcun ulteriore incremento di f_1 .

La procedura continua selezionando progressivamente i nodi con il rapporto beneficio/costo più elevato e quando l'inserimento del successivo nodo selezionato comporterebbe il superamento del budget

$$c(S) > k,$$

l'algoritmo termina e restituisce l'ultimo seed set il cui costo rispettava il vincolo

$$c(S) \leq k.$$

2.4.2 Cost-Seeds-Greedy con f_2

Il secondo algoritmo considerato mantiene invariata la struttura generale di Cost-Seeds-Greedy, ma utilizza una differente funzione obiettivo, indicata con f_2 . Anche in questo caso il seed set viene costruito in maniera iterativa scegliendo, a ogni passo, il nodo che massimizza il rapporto tra guadagno marginale e costo.

Dato il seed set corrente S e un nodo candidato $u \notin S$, il guadagno marginale è definito come

$$f_2(S \cup u) - f_2(S),$$

e il criterio di selezione diventa

$$\frac{\Delta_u f_2(S)}{c(u)}$$

scegliendo quindi

$$\arg \max_{u \in V \setminus S} \frac{\Delta_u f_2(S)}{c(u)}.$$

La funzione f_2 è definita come

$$\sum_{v \in V} \sum_{i=1}^{|N(v) \cap S|} \max \left\{ \left\lceil \frac{d(v)}{2} \right\rceil - i + 1, 0 \right\}$$

dove

$$\left\lceil \frac{d(v)}{2} \right\rceil$$

rappresenta la soglia majority del nodo v .

A differenza di f_1 , nella quale ogni nuovo vicino appartenente al seed set fornisce un incremento unitario fino al raggiungimento della soglia, f_2 assegna un contributo differente a seconda del numero di vicini di v già presenti in S .

La procedura di arresto rimane quella prevista: una volta individuato il nodo con il miglior rapporto beneficio/costo, se il suo inserimento determina il superamento del budget k , l'algoritmo termina restituendo l'ultimo seed set ammissibile.

2.4.3 Cascade-Gain Greedy

Il terzo algoritmo considerato è **Cascade-Gain Greedy (CGG)**, sviluppato nell'ambito del progetto con l'obiettivo di utilizzare direttamente il processo di Majority Cascade come criterio di selezione dei seed.

A differenza delle due varianti di Cost-Seeds-Greedy, che stimano la qualità di un candidato mediante le funzioni f_1 e f_2 , CGG valuta direttamente quanto l'inserimento di un nodo nel seed set aumenti il numero complessivo di nodi attivati.

Sia S il seed set corrente e sia

$$Inf[G, S]$$

l'insieme dei nodi attivati dalla Majority Cascade generata da S . Per ogni candidato

$$u \notin S,$$

viene considerato il nuovo seed set

$$S \cup u$$

e viene valutata la corrispondente diffusione

$$Inf[G, S \cup u].$$

Il guadagno marginale associato al candidato u è quindi definito come

$$|Inf[G, S \cup u]| - |Inf[G, S]|$$

e rappresenta il numero di nuovi nodi che diventerebbero attivi scegliendo u come ulteriore seed.

Poiché i nodi possono presentare costi differenti, il guadagno viene normalizzato rispetto al costo del candidato. Lo score utilizzato dall'algoritmo è quindi

$$\frac{|Inf[G, S]|}{c(u)}$$

e a ogni iterazione viene selezionato il nodo

$$\arg \max_u score_{CGG}(u, S)$$

tra quelli ancora compatibili con il budget residuo.

L'idea alla base dell'algoritmo è quindi quella di privilegiare il nodo che produce il **maggior incremento effettivo** della diffusione per unità di costo. In questo modo vengono considerate non soltanto le attivazioni direttamente collegate al candidato, ma anche tutte le eventuali attivazioni indirette generate dalla cascade.

Algorithm 1: Cascade-Gain Greedy

Input: Grafo non orientato $G = (V, E)$, funzione di costo $c : V \rightarrow \mathbb{N}$, budget k

Output: Seed set $S \subseteq V$ tale che $c(S) \leq k$

```

1  $S \leftarrow \emptyset$ ;
2  $B \leftarrow 0$ ;
3  $A \leftarrow \text{Inf}[G, S]$ ;
4 while  $B < k$  and  $A \neq V$  do
5    $\text{bestNode} \leftarrow \perp$ ;
6    $\text{bestRatio} \leftarrow -\infty$ ;
7    $\text{bestGain} \leftarrow -\infty$ ;
8   foreach  $u \in V \setminus A$  do
9     if  $B + c(u) \leq k$  then
10        $A_u \leftarrow \text{Inf}[G, S \cup \{u\}]$ ;
11        $\text{gain}(u) \leftarrow |A_u| - |A|$ ;
12        $\text{ratio}(u) \leftarrow \frac{\text{gain}(u)}{c(u)}$ ;
13       if  $\text{ratio}(u) > \text{bestRatio}$  or
          $(\text{ratio}(u) = \text{bestRatio} \text{ and } \text{gain}(u) > \text{bestGain})$  then
14          $\text{bestNode} \leftarrow u$ ;
15          $\text{bestRatio} \leftarrow \text{ratio}(u)$ ;
16          $\text{bestGain} \leftarrow \text{gain}(u)$ ;
17   if  $\text{bestNode} = \perp$  then
18     break;
19    $S \leftarrow S \cup \{\text{bestNode}\}$ ;
20    $B \leftarrow B + c(\text{bestNode})$ ;
21    $A \leftarrow \text{Inf}[G, S]$ ;
22 return  $S$ ;
```

Una versione diretta dell'algoritmo richiederebbe di simulare da zero la Majority Cascade per ogni candidato a ogni iterazione. Su una rete di dimensioni elevate questa procedura risulterebbe particolarmente costosa, poiché gran parte delle attivazioni già determinate dal seed set corrente verrebbe ricalcolata ripetutamente.

Per questo motivo l'implementazione utilizza una procedura incrementale. Indicato con

$$A = \text{Inf}[G, S]$$

l'insieme dei nodi già attivi, per valutare un candidato u non è necessario ricostruire l'intera cascade a partire da S . La simulazione può partire direttamente dalla

configurazione già raggiunta, aggiungendo il candidato e propagando solamente le nuove attivazioni che esso rende possibili.

In questo modo viene calcolato direttamente l'incremento

$$|Inf[G, S \cup u]| - |Inf[G, S]|$$

evitando di ripetere la parte del processo già nota.

2.5 Protocollo sperimentale

La procedura sperimentale è stata definita con l'obiettivo di confrontare le tre strategie di selezione del seed set sia in termini di capacità di diffusione sulla rete originale, sia in termini di robustezza rispetto a modifiche della struttura della rete.

Gli esperimenti sono stati condotti considerando entrambe le funzioni di costo descritte nelle sezioni precedenti:

$$c_{\text{random}} \quad \text{e} \quad c_{\text{degree}},$$

e i tre algoritmi:

$$\text{CSG-}f_1, \quad \text{CSG-}f_2, \quad \text{CGG}.$$

Poiché le due funzioni di costo producono distribuzioni di valori molto differenti, non è stato utilizzato lo stesso valore assoluto di budget per entrambe. Il budget è stato invece espresso come percentuale del costo complessivo della rete. Sono stati considerati cinque livelli:

$$p \in \{1\%, 2\%, 5\%, 10\%, 20\%\}$$

Per ciascuna combinazione di funzione di costo, budget e algoritmo viene determinato un seed set S sulla rete originale G . Il numero complessivo di configurazioni baseline è pertanto

$$2 \times 3 \times 5 = 30.$$

Per ogni configurazione vengono memorizzati:

- il seed set S ;
- il numero di seed selezionati;
- il costo effettivo $c(S)$;
- il numero finale di nodi influenzati $|Inf[G, S]|$;
- la percentuale della rete influenzata;
- il numero di nodi attivati dalla cascade oltre ai seed iniziali;
- il numero di round necessari alla stabilizzazione;
- il tempo di esecuzione dell'algoritmo.

La percentuale di rete influenzata è calcolata come

$$\frac{|Inf[G, S]|}{|V|} \cdot 100$$

La seconda fase sperimentale è dedicata alla valutazione della robustezza dei seed set rispetto a modifiche strutturali della rete. Una volta determinato il seed set S sulla rete originale G , esso non viene ricalcolato dopo la perturbazione, in modo da misurare quanto una soluzione costruita sulla configurazione iniziale continui a essere efficace quando la rete cambia. Sono state considerate due differenti tipologie di perturbazione:

- **rimozione casuale di archi**, per simulare la perdita di relazioni tra gli utenti;
- **rimozione casuale di nodi**, per simulare l'abbandono della rete da parte di alcuni utenti.

Per entrambe le tipologie vengono considerati quattro livelli di perturbazione:

$$\boxed{1\%, 5\%, 10\%, 20\%}$$

rispetto, rispettivamente, al numero totale di archi e di nodi della rete originale. Poiché la scelta degli elementi rimossi è casuale, ciascun livello di perturbazione viene ripetuto

$$\boxed{20}$$

volte. I risultati vengono successivamente aggregati attraverso media e deviazione standard, così da ridurre la dipendenza delle conclusioni da una singola realizzazione casuale.

Nel caso della rimozione degli archi, a partire da

$$G = (V, E)$$

viene costruita una rete modificata

$$G' = (V, E')$$

con

$$E' \subset E.$$

Il seed set rimane invariato e viene quindi confrontata la diffusione ottenuta sulla rete originale,

$$|Inf[G, S]|,$$

con quella osservata sulla rete perturbata,

$$|Inf[G', S]|.$$

Nel caso della rimozione dei nodi, invece, indicando con $R \subseteq V$ l'insieme dei vertici eliminati, la rete modificata è

$$G' = G[V \setminus R].$$

Poiché la rimozione è completamente casuale, anche alcuni nodi appartenenti al seed set originale possono essere eliminati. Il seed set effettivamente disponibile nella rete perturbata diventa quindi

$$\boxed{S' = S \cap V(G')}$$

e la diffusione viene valutata attraverso

$$|Inf[G', S']|.$$

Per entrambe le perturbazioni viene utilizzata una misura di **retention**, definita come il rapporto tra l'influenza ottenuta dopo la modifica della rete e quella osservata nella configurazione originale.

Per la rimozione degli archi:

$$\frac{|Inf[G', S]|}{|Inf[G, S]|} \cdot 100$$

mentre, nel caso della rimozione dei nodi:

$$\frac{|Inf[G', S']|}{|Inf[G, S]|} \cdot 100.$$

Un valore pari al 100% indica che il seed set mantiene la stessa efficacia della configurazione originale; valori inferiori indicano una perdita di capacità di diffusione, mentre valori superiori al 100% indicano un incremento dell'influenza dopo la perturbazione.

Capitolo 3

Risultati

In questo capitolo vengono presentati e analizzati i risultati ottenuti dalle diverse configurazioni sperimentali considerate. Il confronto riguarda i tre algoritmi di selezione del seed set, valutati con entrambe le funzioni di costo e per differenti livelli di budget.

L'analisi prende inizialmente in esame l'efficacia dei seed set sulla rete originale, misurata attraverso il numero di nodi complessivamente influenzati al termine della Majority Cascade. Successivamente vengono confrontati i risultati ottenuti dalle diverse strategie, considerando anche i relativi tempi di esecuzione.

Infine, viene analizzata la robustezza dei seed set rispetto a modifiche strutturali della rete, osservando come varia la capacità di diffusione in seguito alla rimozione casuale di archi e nodi. I risultati delle perturbazioni sono riportati mediante valori medi calcolati sulle diverse ripetizioni sperimentali e accompagnati da misure di variabilità.

3.1 Influenza al variare del budget

La prima analisi riguarda l'efficacia dei seed set ottenuti dai tre algoritmi al variare del budget disponibile. Per ciascuna funzione di costo sono stati considerati i livelli di budget pari a

$$1\%, 2\%, 5\%, 10\%, 20\%$$

del costo complessivo della rete.

Per ogni configurazione viene misurato il numero finale di nodi attivati dalla Majority Cascade,

$$|Inf[G, S]|,$$

e la corrispondente percentuale di rete influenzata,

$$\frac{|Inf[G, S]|}{|V|} \cdot 100.$$

L'obiettivo di questa analisi è osservare come la capacità di diffusione evolva all'aumentare delle risorse disponibili e confrontare il comportamento di CSG- f_1 , CSG- f_2 e Cascade-Gain Greedy nelle stesse condizioni sperimentali.

3.1.1 Random Cost Function

La prima analisi considera la funzione di costo casuale, per la quale a ciascun nodo è stato assegnato un valore intero compreso tra 1 e 10. La Figura 3.1 mostra il numero di nodi influenzati dai tre algoritmi al variare del budget disponibile, mentre la Tabella 3.1 riporta i corrispondenti valori percentuali rispetto ai 4039 nodi della rete.

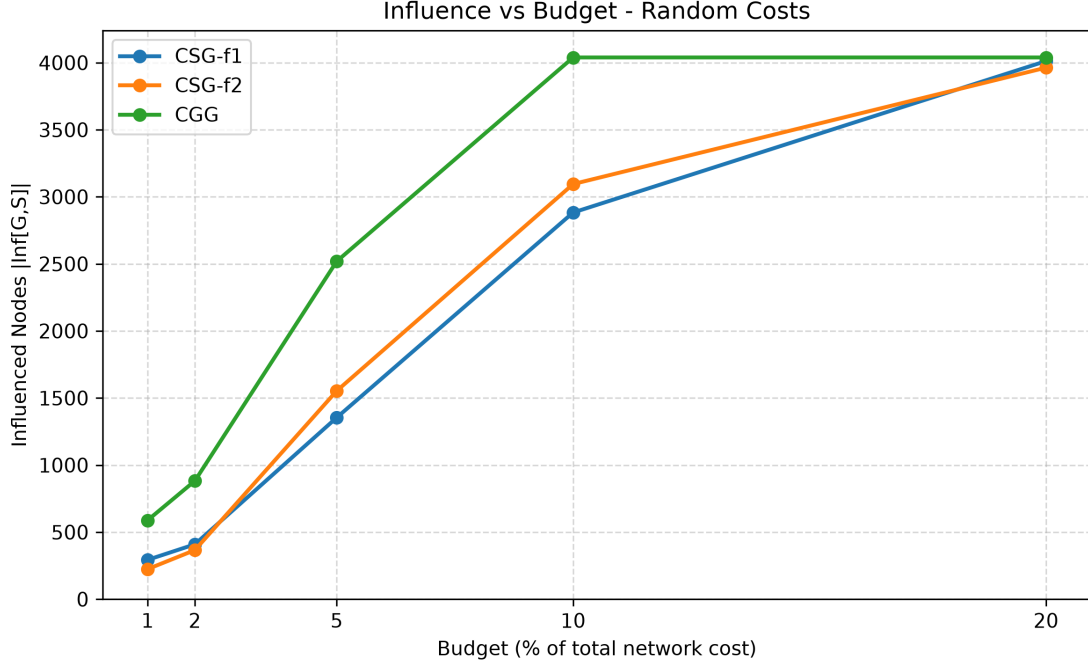


Figura 3.1: Numero di nodi influenzati al variare del budget con Random Cost Function.

Tabella 3.1: Percentuale di nodi influenzati al variare del budget con Random Cost Function.

Budget (%)	CSG- f_1 (%)	CSG- f_2 (%)	CGG (%)
1	7.25	5.55	14.56
2	10.13	9.06	21.84
5	33.52	38.45	62.37
10	71.38	76.63	100.00
20	99.33	98.12	100.00

I risultati evidenziano innanzitutto un aumento della capacità di diffusione al crescere del budget per tutte le strategie considerate. Cascade-Gain Greedy ottiene il risultato migliore in tutte le configurazioni, con un vantaggio particolarmente evidente ai budget più contenuti. Con il 10% del costo totale, CGG produce invece una diffusione completa,

$$|Inf[G, S]| = 4039,$$

raggiungendo quindi il 100% dei nodi, mentre le due varianti di Cost-Seeds-Greedy mostrano risultati più vicini tra loro.

È inoltre importante osservare che, nel caso della Random Cost Function, il costo assegnato a un nodo è indipendente dalle sue caratteristiche strutturali. Un nodo molto connesso e potenzialmente rilevante per la diffusione può quindi avere un costo basso, mentre un nodo poco connesso può risultare relativamente costoso. Questo rende possibile, in alcune configurazioni, selezionare nodi strutturalmente vantaggiosi senza sostenere necessariamente un costo elevato e contribuisce a spiegare la rapida crescita dell'influenza osservata all'aumentare del budget.

3.1.2 Degree Cost Function

Nel secondo scenario è stata utilizzata la funzione costo basata sul grado, che assegna un costo più alto ai nodi con grado elevato. In questo modo, i nodi più connessi, che potrebbero essere utili per diffondere l'influenza, diventano anche più costosi da selezionare come seed. La Figura 3.2 mostra il numero di nodi influenzati dai tre algoritmi utilizzando la Degree Cost Function, mentre la Tabella 3.2 riporta i corrispondenti valori percentuali rispetto alla dimensione complessiva della rete.

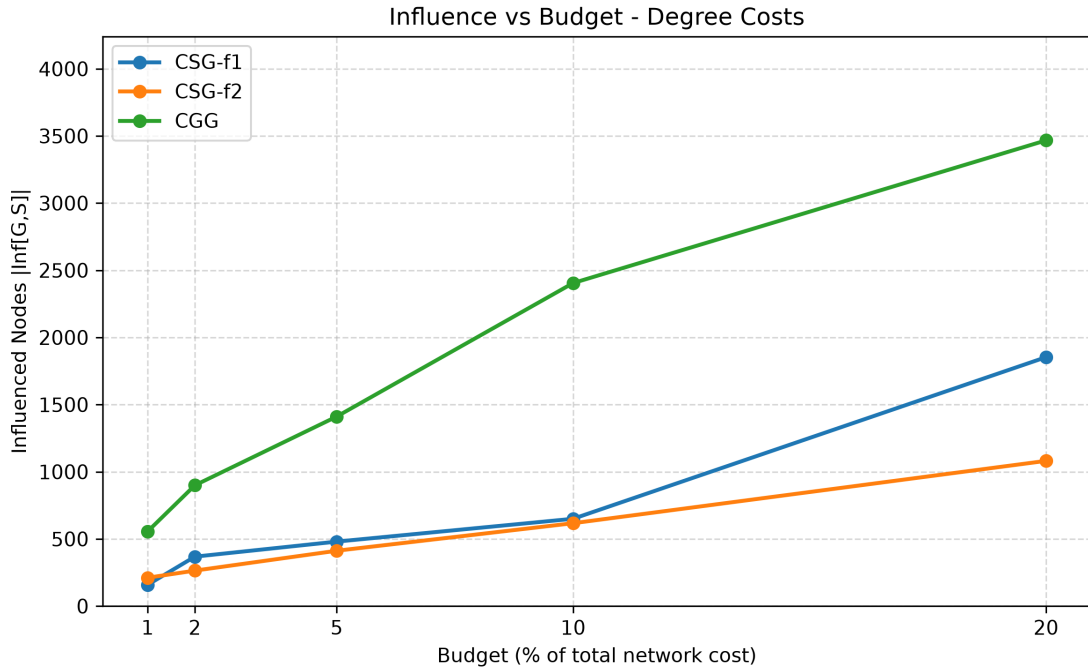


Figura 3.2: Numero di nodi influenzati al variare del budget con Degree Cost Function.

Tabella 3.2: Percentuale di nodi influenzati al variare del budget con Degree Cost Function.

Budget (%)	CSG- f_1 (%)	CSG- f_2 (%)	CGG (%)
1	3.91	5.22	13.74
2	9.11	6.54	22.26
5	11.88	10.20	34.93
10	16.09	15.28	59.54
20	45.88	26.76	85.84

Anche in questo caso l'aumento del budget determina una crescita della diffusione per tutti gli algoritmi, ma l'andamento risulta sensibilmente diverso rispetto alla Random Cost Function. Cascade-Gain Greedy mantiene il risultato migliore per tutti i livelli di budget, mentre tra le due varianti CSG- f_2 ottiene un risultato migliore solamente con il budget minimo.

La minore diffusione rispetto al caso Random è strettamente legata alla natura della Degree Cost Function. Poiché

$$\left\lceil \frac{d(u)}{2} \right\rceil,$$

i nodi maggiormente connessi risultano anche più costosi. La selezione di nodi potenzialmente strategici per la propagazione richiede quindi una quota maggiore del budget, limitando la possibilità di includerne contemporaneamente molti nel seed set. Questo effetto è particolarmente evidente ai budget più contenuti e rende più difficile raggiungere una diffusione completa della rete.

3.2 Confronto complessivo

Il confronto complessivo dei risultati consente di evidenziare differenze nette nel comportamento dei tre algoritmi considerati.

Le due varianti CSG- f_1 e CSG- f_2 mostrano prestazioni relativamente vicine, ma nessuna delle due risulta sistematicamente superiore all'altra in tutte le configurazioni.

Il comportamento di Cascade-Gain Greedy risulta invece molto più uniforme. CGG ottiene la maggiore diffusione in tutte le configurazioni considerate, indipendentemente dalla funzione di costo e dal livello di budget. Il vantaggio è particolarmente evidente nei budget bassi e intermedi, dove la capacità di selezionare nodi in grado di generare vere cascade consente di ottenere un numero di attivazioni significativamente superiore rispetto alle due varianti di Cost-Seeds-Greedy.

Nel complesso, i risultati mostrano quindi che CGG è l'algoritmo più efficace

dal punto di vista della massimizzazione dell'influenza, mentre le due varianti di Cost-Seeds-Greedy presentano prestazioni più dipendenti dalla configurazione sperimentale. Tale superiorità in termini di qualità della soluzione deve tuttavia essere valutata insieme al maggiore costo computazionale richiesto da CGG, aspetto analizzato nella sezione successiva.

3.3 Analisi dei tempi di esecuzione

Oltre alla qualità dei seed set ottenuti, è stato analizzato anche il tempo di esecuzione richiesto dai tre algoritmi. Per fornire un confronto sintetico, per ciascun algoritmo è stato calcolato il tempo medio sulle dieci configurazioni considerate, ottenute combinando le due funzioni di costo con i cinque livelli di budget.

Tabella 3.3: Tempo medio di esecuzione dei tre algoritmi.

Algoritmo	Tempo medio (s)
CSG- f_1	5.48
CSG- f_2	8.35
CGG	24.38

Come mostrato in Tabella 3.3, CSG- f_1 risulta l'algoritmo più rapido, con un tempo medio di circa 5.48 secondi, seguito da CSG- f_2 con 8.35 secondi. Cascade-Gain Greedy presenta invece un costo computazionale sensibilmente superiore, con un tempo medio di circa 24.38 secondi.

La differenza deriva direttamente dal criterio utilizzato per valutare i candidati. Le due varianti di Cost-Seeds-Greedy calcolano il guadagno marginale attraverso informazioni locali relative ai vicinati e alle soglie dei nodi. CGG, invece, valuta direttamente l'incremento prodotto sulla Majority Cascade,

$$|Inf[G, S \cup u]| - |Inf[G, S]|,$$

richiedendo quindi una procedura di valutazione più onerosa.

L'ottimizzazione incrementale introdotta nell'implementazione di Cascade-Gain Greedy ha comunque permesso di ridurre significativamente il costo computazionale, mantenendo i tempi compatibili con l'esecuzione dell'intera campagna sperimentale.

3.4 Valutazione della robustezza

Dopo aver analizzato l'efficacia dei tre algoritmi sulla rete originale, è stata valutata la robustezza dei seed set rispetto a modifiche strutturali della rete. L'obiettivo di questa fase è verificare quanto una soluzione determinata sulla configurazione iniziale G continui a essere efficace quando la rete evolve, come previsto dalla natura dinamica delle online social networks.

3.4.1 Rimozione casuale di archi

Il primo esperimento di robustezza valuta l'effetto della rimozione casuale di archi sull'efficacia dei seed set determinati sulla rete originale. Per ciascuna configurazione sono stati rimossi rispettivamente l'1%, il 5%, il 10% e il 20% degli archi, mantenendo invariato il seed set S . Ogni livello di perturbazione è stato ripetuto 20 volte; nei grafici vengono pertanto riportati i valori medi di

$$|Inf[G', S]|$$

e la relativa deviazione standard.

Le Figure 3.3 e 3.4 mostrano rispettivamente i risultati ottenuti con Random Cost e Degree Cost. Il punto corrispondente allo 0% rappresenta la diffusione sulla rete originale e costituisce quindi la baseline rispetto alla quale valutare le successive perturbazioni.

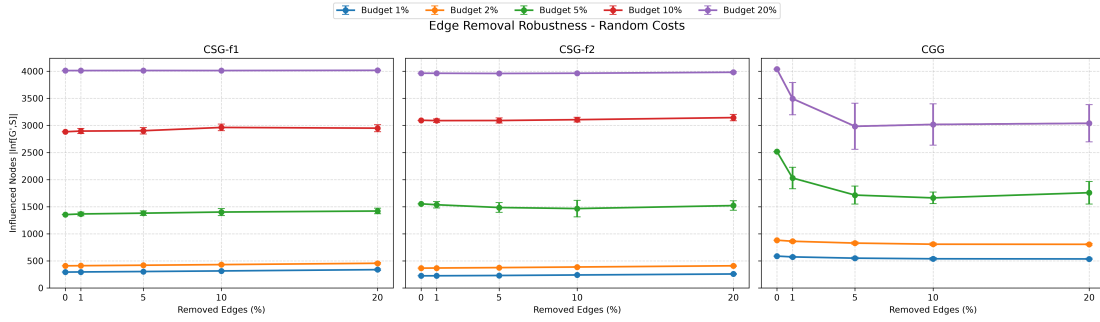


Figura 3.3: Robustezza alla rimozione casuale di archi con Random Cost Function. Le barre di errore rappresentano la deviazione standard sulle 20 ripetizioni.

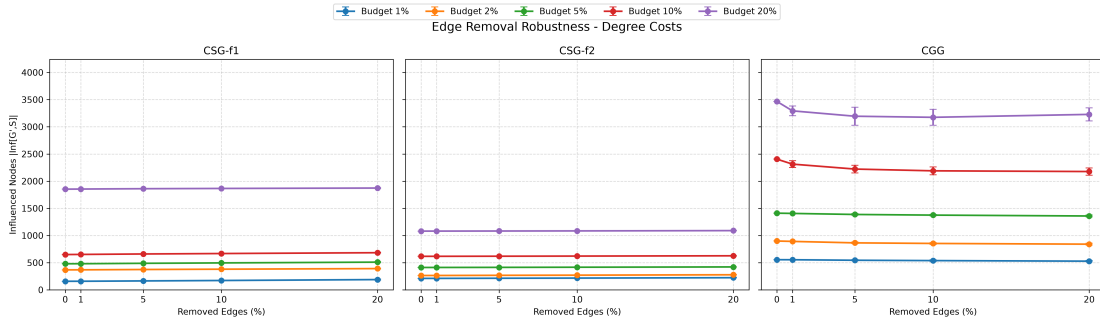


Figura 3.4: Robustezza alla rimozione casuale di archi con Degree Cost Function. Le barre di errore rappresentano la deviazione standard sulle 20 ripetizioni.

Un primo risultato rilevante riguarda le due varianti di Cost-Seeds-Greedy. La loro diffusione risulta generalmente molto stabile e, in numerose configurazioni, aumenta addirittura all'aumentare della percentuale di archi rimossi. Tale comportamento è una conseguenza della stessa definizione del modello Majority. La rimozione di un arco non produce solamente una perdita di connettività, ma può ridurre il grado di un nodo e, di conseguenza, la sua soglia.

Il comportamento di Cascade-Gain Greedy è differente. Poiché CGG costruisce il seed set valutando direttamente le cascade generate sulla topologia originale, le sue soluzioni risultano maggiormente dipendenti dalla presenza degli archi che hanno consentito quelle propagazioni. La loro rimozione tende pertanto a ridurre l'efficacia della soluzione.

La Tabella 3.4 riassume la retention osservata nel caso di perturbazione più intensa, corrispondente alla rimozione del 20% degli archi. I valori superiori al 100%) indicano che la diffusione sulla rete perturbata risulta maggiore rispetto a quella osservata sulla rete originale.

Tabella 3.4: Retention media (%) dopo la rimozione del 20% degli archi.

Budget	Random Cost			Degree Cost		
	CSG- f_1	CSG- f_2	CGG	CSG- f_1	CSG- f_2	CGG
1%	115.44	115.27	91.12	119.78	106.16	95.13
2%	111.44	111.87	91.34	106.37	105.21	93.52
5%	104.86	97.91	69.80	106.24	102.40	96.31
10%	102.29	101.58	75.24	105.18	101.60	90.52
20%	100.11	100.44	75.24	101.09	100.85	93.12

Nel complesso, l'esperimento mostra che la rimozione degli archi non ha un effetto univocamente negativo sulla Majority Cascade. CSG- f_1 e CSG- f_2 risultano particolarmente stabili e possono persino beneficiare della riduzione delle soglie, mentre CGG mantiene generalmente un numero assoluto di nodi influenzati superiore ma mostra una maggiore sensibilità alle modifiche della topologia

3.4.2 Rimozione casuale di nodi

Il secondo esperimento di robustezza considera la rimozione casuale di nodi dalla rete. Rispetto alla rimozione degli archi, questa perturbazione risulta più invasiva, poiché l'eliminazione di un vertice comporta anche la rimozione di tutti i suoi archi incidenti e può coinvolgere direttamente nodi appartenenti al seed set originario.

Indicando con $R \subseteq V$ l'insieme dei nodi rimossi, la rete perturbata è definita come

$$G' = G[V \setminus R].$$

Poiché i seed non vengono protetti dalla perturbazione, il seed set effettivamente disponibile sulla nuova rete diventa

$$S' = S \cap V(G').$$

Le Figure 3.5 e 3.6 mostrano il numero medio di nodi influenzati dopo la rimozione rispettivamente dell'1%, 5%, 10% e 20% dei vertici. Anche in questo caso ogni configurazione è stata ripetuta 20 volte e le barre di errore rappresentano la deviazione standard.

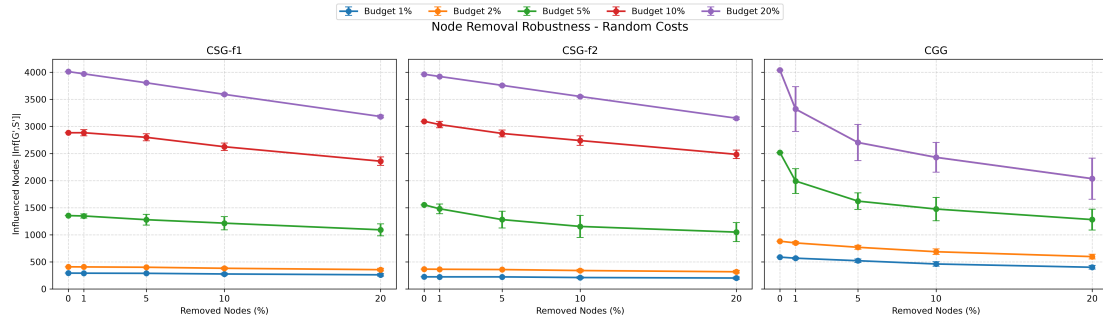


Figura 3.5: Robustezza dei seed set alla rimozione casuale di nodi con Random Cost Function. Le barre di errore rappresentano la deviazione standard sulle 20 ripetizioni.

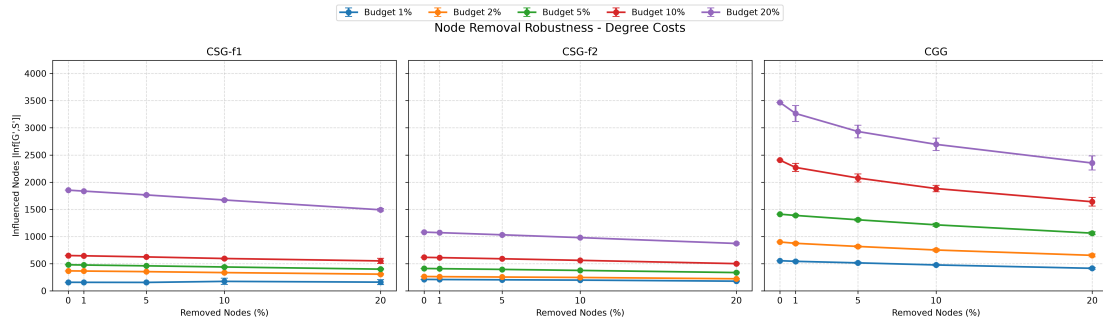


Figura 3.6: Robustezza dei seed set alla rimozione casuale di nodi con Degree Cost Function. Le barre di errore rappresentano la deviazione standard sulle 20 ripetizioni.

A differenza di quanto osservato nella rimozione degli archi, in questo caso emerge generalmente un progressivo deterioramento della capacità di diffusione all'aumentare della percentuale di nodi eliminati. Tale comportamento è prevedibile, poiché oltre alla modifica della topologia viene ridotta direttamente la dimensione della rete e possono essere rimossi alcuni dei seed originariamente selezionati.

Per sintetizzare il comportamento nelle condizioni di perturbazione più severa, la Tabella 3.5 riporta la retention media ottenuta con la rimozione del 20% dei nodi.

Tabella 3.5: Retention media (%) dopo la rimozione del 20% degli nodi.

Budget	Random Cost			Degree Cost		
	CSG- f_1	CSG- f_2	CGG	CSG- f_1	CSG- f_2	CGG
1%	89.22	90.13	68.13	101.71	84.62	74.77
2%	87.04	86.71	67.72	83.52	84.22	72.72
5%	80.65	67.55	50.83	83.24	81.67	75.25
10%	81.80	80.28	50.40	84.78	81.18	68.17
20%	79.33	79.52	50.40	80.49	80.67	67.84

La tabella evidenzia come, al livello massimo di perturbazione, CSG- f_1 e CSG- f_2 mantengano generalmente una quota maggiore dell'influenza originaria rispetto a CGG.

Nel complesso, la rimozione dei nodi risulta più penalizzante della rimozione dei soli archi. CSG- f_1) e CSG- f_2 presentano generalmente una maggiore retention relativa, mentre CGG continua a produrre i valori assoluti di influenza più elevati ma mostra una maggiore sensibilità alle perturbazioni, soprattutto con Random Cost. Anche in questo caso emerge quindi una distinzione tra efficacia della soluzione sulla rete originale e capacità di conservarne le prestazioni quando la struttura della rete viene modificata.

3.5 Discussione finale

I risultati ottenuti evidenziano differenze significative tra i tre algoritmi considerati. Cascade-Gain Greedy risulta sistematicamente il metodo più efficace nel massimizzare la diffusione sulla rete originale, soprattutto per budget bassi e intermedi. Tale vantaggio deriva dalla valutazione diretta dell'incremento della Majority Cascade, mentre CSG- f_1 e CSG- f_2 utilizzano funzioni surrogate e mostrano prestazioni più dipendenti dalla specifica configurazione sperimentale.

Anche la funzione di costo influenza sensibilmente i risultati. Con Random Cost, nodi strutturalmente rilevanti possono essere selezionati anche a costo ridotto, favorendo una crescita più rapida dell'influenza. Con Degree Cost, invece, i nodi maggiormente connessi risultano anche più costosi e la diffusione risulta generalmente più limitata.

Dal punto di vista computazionale emerge un chiaro compromesso tra efficacia e tempo di esecuzione. CSG- f_1) e CSG- f_2 risultano più rapidi, mentre CGG richiede tempi superiori ma produce seed set decisamente più efficaci.

Gli esperimenti di robustezza mostrano inoltre che efficacia iniziale e stabilità non coincidono necessariamente. CGG mantiene generalmente il maggiore numero assoluto di nodi influenzati, ma in alcune configurazioni presenta una riduzione

relativa più marcata dopo la perturbazione. Le due varianti CSG mostrano invece spesso una retention maggiore.

Un comportamento particolarmente interessante emerge nella rimozione degli archi: la diffusione può talvolta aumentare perché la riduzione del grado determina anche una diminuzione della soglia majority. La rimozione dei nodi risulta invece complessivamente più penalizzante, poiché può eliminare direttamente anche elementi del seed set e modificare più profondamente la struttura della rete.

Nel complesso, Cascade-Gain Greedy rappresenta la strategia più efficace in termini di massimizzazione dell'influenza, mentre le due varianti di Cost-Seeds-Greedy offrono un compromesso più favorevole in termini di costo computazionale e, in alcune condizioni, di robustezza relativa.

Capitolo 4

Conclusioni

Il progetto ha analizzato il problema della selezione di seed set per un processo di **Majority Influence Diffusion** in una online social network, considerando sia l'efficacia delle soluzioni sulla rete originale sia la loro robustezza rispetto a modifiche strutturali.

Gli esperimenti sono stati condotti sulla rete Facebook utilizzando due differenti funzioni di costo, *Random Cost* e *Degree Cost*, e confrontando tre strategie di selezione: *CSG- f_1* , *CSG- f_2* e *Cascade-Gain Greedy*.

I risultati mostrano che l'aumento del budget produce, come atteso, una maggiore capacità di diffusione per tutti gli algoritmi. La funzione di costo ha inoltre un ruolo determinante: con *Random Cost* risulta più semplice selezionare nodi strutturalmente rilevanti a costo contenuto, mentre *Degree Cost* penalizza maggiormente i nodi ad alto grado e rende quindi il problema di selezione più restrittivo.

Tra gli algoritmi analizzati, *Cascade-Gain Greedy* ha ottenuto i migliori risultati in termini di influenza in tutte le configurazioni considerate. Con *Random Cost*, in particolare, l'algoritmo è riuscito ad attivare l'intera rete già con un budget pari al 10% del costo complessivo. Tale maggiore efficacia è accompagnata da un costo computazionale superiore rispetto alle due varianti di *Cost-Seeds-Greedy*, evidenziando un compromesso tra qualità della soluzione e tempo di esecuzione.

Gli esperimenti di perturbazione hanno inoltre mostrato che la robustezza non dipende esclusivamente dall'efficacia iniziale del seed set. La rimozione degli archi può in alcuni casi favorire la diffusione, poiché la riduzione del grado comporta anche un abbassamento delle soglie majority. La rimozione dei nodi risulta invece generalmente più penalizzante, soprattutto quando coinvolge direttamente elementi appartenenti al seed set.

Nel complesso, l'analisi evidenzia come la selezione dei seed debba essere valutata considerando non soltanto la capacità di massimizzare

$$|Inf[G, S]|$$

sulla rete originale, ma anche la stabilità della soluzione in presenza di cambiamenti nella topologia. *Cascade-Gain Greedy* rappresenta l'approccio più efficace tra quelli considerati, mentre *CSG- f_1* e *CSG- f_2* offrono soluzioni computazionalmente più

economiche e, in alcune configurazioni, caratterizzate da una maggiore robustezza relativa.

Possibili sviluppi futuri potrebbero includere l'analisi su ulteriori online social networks, l'introduzione di differenti modelli di costo e lo studio di strategie che tengano conto esplicitamente della robustezza della soluzione già durante la fase di selezione del seed set.